# Pattern Binding & Regex Operators

Perl 5.14+ & PCRE

| PATTERN / SYNTAX | DESCRIPTION & EXAMPLE |
|---|---|
| `$str =~ /pattern/` | Pattern binding match operator — tests if $str matches regular expression (returns true/false in scalar context)<br>**EXAMPLE:** `if ($email =~ /^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$/i) { ... }` |
| `$str !~ /pattern/` | Negative pattern binding match operator — returns true if $str does NOT match regular expression<br>**EXAMPLE:** `if ($password !~ /\d/) { die "Password needs a digit"; }` |
| `$str =~ s/pattern/replacement/flags` | Substitution operator — replaces matched pattern with replacement string (flags: /g, /i, /e, /r)<br>**EXAMPLE:** `$text =~ s/\bapple\b/orange/gi;` |
| `$str =~ tr/SEARCH/REPLACE/flags` | Transliteration operator (synonym: y///) — translates all characters in search list to replace list (/d delete, /s squash, /c complement)<br>**EXAMPLE:** `$dna =~ tr/atcg/tagc/; # Transliterate DNA bases` |
| `qr/pattern/flags` | Quote regex operator — compiles pattern and flags into a reusable, pre-compiled Regexp object<br>**EXAMPLE:** `my $rx = qr/\b[A-Z]{3}-\d{4}\b/i;` |
| `split(/pattern/, $str [, limit])` | Splits $str into a list of substrings separated by delimiter matches<br>**EXAMPLE:** `my @fields = split(/[,;]\s*/, $csv_line);` |
| `my @matches = ($str =~ /pattern/g)` | List context match with /g flag — returns all captured groups or full matches as a list<br>**EXAMPLE:** `my @words = ($text =~ /\b\w+\b/g);` |
| `while ($str =~ /pattern/g) { ... }` | Scalar context match in while loop — iterates match by match through string, tracking offset via pos($str)<br>**EXAMPLE:** `while ($log =~ /(\d{4}-\d{2}-\d{2})/g) { say "Found date at " . pos($log); }` |

# Special Regex Variables & Pointers

Perl 5.14+ & PCRE

| PATTERN / SYNTAX | DESCRIPTION & EXAMPLE |
|---|---|
| `$1, $2, $3 ...` | Contains the text captured by the 1st, 2nd, 3rd parenthesized capturing group from last successful match<br>**EXAMPLE:** `if ($str =~ /(\w+):(\d+)/) { my ($k, $v) = ($1, $2); }` |
| `$& (or ${^MATCH})` | The entire string matched by the last successful pattern (use /p flag or ${^MATCH} to avoid performance penalty)<br>**EXAMPLE:** `$str =~ /\d+/; say "Matched: $&";` |
| `$` and `$' (or ${^PREMATCH}, ${^POSTMATCH})` | The string preceding ($`) and following ($') the last successful match<br>**EXAMPLE:** `$str =~ /:/; say "Before: $`, After: $'";` |
| `$+ (or ${^LAST_PAREN_MATCH})` | Returns the text matched by the highest-numbered (last) capturing group<br>**EXAMPLE:** `$str =~ /(foo)\|(bar)\|(baz)/; say "Last: $+";` |
| `%+ and %-` | Hashes holding named capture groups: $+{name} contains captured text; $-{name} contains arrayref of all captures for that name<br>**EXAMPLE:** `if ($str =~ /(?<year>\d{4})-(?<month>\d{2})/) { say $+{year}; }` |
| `@- and @+` | Arrays holding start ($-) and end ($+) byte/character offsets in target string for $& (index 0) and capture groups (indices 1..N)<br>**EXAMPLE:** `my $start = $-[0]; my $end = $+[0]; # Offsets of full match` |
| `pos($str)` | Returns or sets the current offset position where the next /g search will resume<br>**EXAMPLE:** `pos($str) = 0; # Reset regex cursor` |
| `\K` | Keep-out escape sequence: resets match start; drops everything matched to the left from $& and s/// replacement<br>**EXAMPLE:** `$str =~ s/foo\Kbar/baz/; # replaces only "bar" when preceded by "foo"` |

# Pattern Modifiers & Flags

Perl 5.14+ & PCRE

| PATTERN / SYNTAX | DESCRIPTION & EXAMPLE |
|---|---|
| `/g` | Global — match or replace all occurrences throughout the target string<br>**EXAMPLE:** `$str =~ s/\s+/ /g;` |
| `/i` | Case-insensitive matching (ignores uppercase vs lowercase distinction)<br>**EXAMPLE:** `$str =~ /hello/i → matches "HELLO", "Hello", "hello"` |
| `/m` | Multiline mode — ^ and $ match beginning and end of each physical line within string<br>**EXAMPLE:** `$str =~ /^#\s*/m;` |
| `/s` | Single-line / Dot-all mode — permits the dot (.) metacharacter to match newline (\n)<br>**EXAMPLE:** `$str =~ /<div.*?>(.*?)<\/div>/s;` |
| `/x and /xx` | Extended / Verbose mode — ignores unescaped whitespace and permits # comments; /xx also ignores whitespace inside bracket classes<br>**EXAMPLE:** `$str =~ / \d{4} # Year \n -\d{2} # Month /x;` |
| `/e` | Evaluate replacement string as executable Perl code expression (can be chained e.g. /ee)<br>**EXAMPLE:** `$str =~ s/(\d+)/$1 * 2/ge; # Doubles all integers` |
| `/r` | Non-destructive substitution (Perl 5.14+) — returns modified copy of string without altering original variable<br>**EXAMPLE:** `my $slug = $title =~ s/[^a-z0-9]+/-/gir;` |
| `/a and /aa` | ASCII-only matching rules for \d, \w, \s, and POSIX classes; /aa enforces ASCII casing<br>**EXAMPLE:** `$str =~ /\w+/a;` |
| `/u` | Unicode rules — enables full Unicode character semantics on character classes<br>**EXAMPLE:** `$str =~ /\w+/u;` |
| `/o` | Compile pattern only once when compiling the script (use with caution if variable interpolation changes)<br>**EXAMPLE:** `$str =~ /$pattern/o;` |

# Advanced Constructs & PCRE Features

Perl 5.14+ & PCRE

| PATTERN / SYNTAX | DESCRIPTION & EXAMPLE |
|---|---|
| `(?<name>pattern) / (?'name'pattern)` | Named capture group — captures matched substring and stores it in $+{name}<br>EXAMPLE: `(?<user>\w+)@(?<host>[\w.-]+)` |
| `\g{1}, \g{-1}, \g{name}, \k<name>` | Backreferences by absolute number (\g{1}), relative position (\g{-1} for previous group), or name (\k<name>)<br>EXAMPLE: `(["'])(.*?)\g{-2}` |
| `(?>pattern) and *+, ++, ?+` | Atomic grouping and possessive quantifiers — matches greedily and prevents backtracking into the group<br>EXAMPLE: `(?>\w+):` |
| `(?\|pattern1\|pattern2)` | Branch reset group — resets capturing group numbering across alternatives so each branch uses the same $1, $2<br>EXAMPLE: `(?\|(\d{4})-(\d{2})\|([a-z]+)/(\d+))` |
| `(?(condition)yes-pattern\|no-pattern)` | Conditional match — if group number/name or lookaround condition evaluates true, matches yes-pattern, else no-pattern<br>EXAMPLE: `^(?<paren>\()?(\d{3})(?(paren)\)\|-)(\d{4})$` |
| `(?R) or (?0) / (?1) / (?&name)` | Recursive regex and subroutine calls — (?R) recurses the entire pattern; (?1) recurses group 1; (?&name) recurses named group<br>EXAMPLE: `\((?:[^()]\|(?R))*\)` |
| `(*SKIP)(*FAIL) / (*SKIP)(*F)` | Backtracking control verbs — discards unwanted matches and forces regex engine to skip ahead<br>EXAMPLE: `"[^"]*"(*SKIP)(*F)\|\bkeyword\b` |
| `(?{ code }) and (??{ code })` | Embedded code blocks — (?{ code }) executes arbitrary Perl code at match point; (??{ code }) evaluates code and treats result as sub-pattern<br>EXAMPLE: `(\w+)(?{ say "Saw $^N" })` |

# Perl Character Classes & Unicode Properties

Perl 5.14+ & PCRE

| PATTERN / SYNTAX | DESCRIPTION & EXAMPLE |
|---|---|
| `\p{Letter} / \p{L} / \p{Lu} / \p{Ll}` | Unicode General Category: all letters (\p{L}), uppercase letter (\p{Lu}), lowercase letter (\p{Ll})<br>EXAMPLE: `\p{Lu}\p{Ll}+` |
| `\p{Number} / \p{N} / \p{Nd}` | Unicode numbers: all numbers (\p{N}) or decimal digits (\p{Nd})<br>EXAMPLE: `\p{N}+` |
| `\p{Punctuation} / \p{P}` | Unicode punctuation characters: open (\p{Ps}), close (\p{Pe}), connector (\p{Pc}), dash (\p{Pd})<br>EXAMPLE: `\p{P}` |
| `\P{Property}` | Negated Unicode property — matches any character that does NOT possess the property<br>EXAMPLE: `\P{L}+` |
| `\p{Script=Greek} / \p{Script=Latin}` | Unicode Script matching (e.g. \p{Script=Arabic}, \p{Script=Cyrillic}, \p{Script=Han})<br>EXAMPLE: `\p{Script=Greek}+` |
| `\X` | Extended Unicode grapheme cluster — matches a complete user-perceived character including combining marks/accents<br>EXAMPLE: `\X` |